NOVA SCHOOL OF SCIENCE & TECHNOLOGY

Arquitetura de Computadores 2025/26

Lab 8

Topics: *Functions/subroutines in RISC-V assembly language.*

Part I – Arguments and return values in functions

Observation: *The goal of this section is to adapt some of the code of the previous class into functions/subroutines according to the function calling convention in RISC-V assembly language.*

1. Complete the program below in RISC-V assembly language that follows the convention for function invocation and implements a subroutine/function to add two integers. The two values have to be passed as arguments to the function in registers *a0* and *a1*, and the result is returned in register *a0*.

```
..data
    var1:   .word 35 # example values (try with different values)
    var2:   .word 10
    result: .word 0  # must contain the addition result of var1 and var2

.text
.global _start
_start:

    ??            # first argument must be in a0
    ??            # second argument must be in a1
    call addnum   # the same as: jal addnum -- ra stores the return address
    ??            # store the returned value in variable result

    # syscall to write an integer
    # li a0, value     # value to write
    li a7, 1
    ecall

    # exit(int status)
    li a0, 0          # exit status
    li a7, 93         # syscall number for exit
    ecall

# a0: first argument
# a1: second argument
# a0: return value
addnum:
    add a0, a0, a1
    ret             # the same as: jr ra  -- ra has to have the return address
```

2. Write a program in RISC-V assembly language that implements a subroutine/function that receives two numbers as arguments and returns the biggest of the two. The two arguments are passed to the function according to the convention of the RISC-V assembly language, and the same applies to the return value. Your code has to invoke the function with the values of two integers in memory and store the result in a third variable named *result*. As a starting point, use the following data segment:

```
.data
  var1:   .word 35  # example values (try with different values)
  var2:   .word 10
  result: .word 0   # must contain the addition result of var1 and var2
```

3. Adapt your code from the previous example (and from exercise 5 in lab 7) to write a new function/subroutine named `max_elem` that identifies the bigest element in a vector of integers. The function's first argument is the vector's address, the second argument is the number of integers in the vector, and the function returns the vector's biggest element. As a starting point use the following data segment, invoke your function and store its returned value into the variable `max`, and then execute your program in stepwise manner:

```
.data
  vector: .word -1, 5, 1, 1, 4
  length: .byte 5 # number of integers in the vector
     max: .word 0   # must contain the sum of all elements in vector
```

4. Recall exercise number 2 from lab 7, where you were asked to write a program that transforms a string (a sequence of characters terminated with the null character '\0') to its equivalent in caps. Adapt your program to define a function/subroutine called *`caps`* (or *maiusculas*) that receives as argument the address of a string and transform all its characters to their caps equivalent. Use that function to transform string1 and string2 for the data segment bellow. Assuming your data segment is

```
.data
  msg1: .ascii "Hello world! How are you today?\n\0"
  msg2: .ascii "Tuesdays are sometimes Fridays...\n\0"
```

the output of your program should be:

```
Hello world! How are you today?
HELLO WORLD! HOW ARE YOU TODAY?
Tuesdays are sometimes Fridays…
TUESDAYS ARE SOMETIMES FRIDAYS…
-- program is finished running (0) --
```

5. Adapt exercise number 6 from lab 7 and define a subroutine named `add_vectors` that adds the contents of two vectors and saves the result in a third one. The function receives as the first two arguments the addresses of the vectors to be added, the third argument is an integer with the number of elements in the vector, and the fourth argument is the address of the vector with the result. The prototype in C might be *`add_vectors(int v1[], int v2[], int len, int vres[])`*. As a starting point, use the following data segment and invoque your function to produce the result into the vector `vecresult`:

```
.data
  vector1:  .word -1, 5, 1, 1, 4
  vector2:  .word 1, 3, -1, 5, 9
  vecresult: .space 20  # number of bytes for 5 integers
  length: .byte 5   # number of integers in the vectors
```

6. Adapt exercise number 7 from lab 7 to write a function named sub_*`vectors`* that subtracts the contents of two vectors of bytes and saves the result in the first vector (`vector1`) . Use the following as a starting point, and you have to calculate the dimension of `vector1` using its start and end addresses, i.e. the address of the next varaible. The C prototype might be *`void sub_vect(char * v1[], char * v2, int len).`*

```
.data
  vector1:  .byte -1, 5, 1, 1, 4, 20, -7
  vector2:  .byte 1, 3, -1, 5, 9, -5, 0
  # in the final, vector1 shoud have: -2, 2, 2, -4, -5, 25, -7
```

Parte II – other subroutines

Observation: *The goal of this section is to exercise further the function calling convention in RISC-V assembly.*

7. Implement in RISC-V assembly a subroutine equivalent to the following prototype in the C language: `unsigned char lower_vowel(unsigned char *str)` that returns the vowel with the lowest ascii present in the string. For instance, when invoked with `lower_vowel("Hello world!")` the function returns '*e*'. Implement a program to test your function.

8. Implement in RISC-V assembly a subroutine equivalent to the following prototype in the C language: void `integer_address(int vec[], int len, int value, int *addr)` that searches the vector for the presence of the `value` in a vector and returns its absolute address in the fourth argument `addr` (i.e *if vec[i] == value, addr = &vec[i]*) . If the value is not found, `addr` receives 0.

9. Implement in RISC-V assembly a subroutine equivalent to the following prototype in the C language: `int integer_offset(int vec[], int len, int value)` that searches the vector for the presence of the `value` in the vector and returns its offset in bytes from the begining of the vector (i.e *if vec[i] == value, return &vec[i]-&vec[0]*) . If the value is not present the function returns -1.

10. Write in RISC-V assembly a subroutine to compare lexicograpically two strings A and B (i.e. similarly to *strcmp(char *A, char * B)*). The comprison is left to right and a string A is considered to be bigger lexicograpically than string B, if the first different character in A has a bigger ASCII code than the character in the same position in B (the string B may have been finished as well). If the strings are equal, the function returns zero; if the first string A is smaller that B, the result is -1; otherwise the function returns 1. Use the following examples as test:

```
.data
  cad1:   .ascii   "hellu\n\0"
  cad2:   .ascii   "hello\n\0"
  res:    .int       0         # the result is 1

.data
  cad1:   .ascii   "flor\n\0"
  cad2:   .ascii   "floresta\n\0"
  res:    .int       0         # the result is -1
```

References

[1] The convention for calling functions' arguments and return values in RISC-V assembly is describe in the slides of the theoretical classes.

[2] The RARS simulator is available in clip or can be downloaded from:

https://github.com/ajccosta/rars/raw/refs/heads/master/rars.jar

Execute the simulator in the command line with*:*

```
$ java -jar rars.jar
```

[3] Two cheat sheets for the RISC-V assembly and the RISC-V book are available in *clip,* in section "*Textos de Apoio*" and in Piazza.

[4] The ASCII table is available from:

https://www.ascii-code.com/